SOBRACERO

Guía funcional y técnica de la aplicación

Explicación de producto, arquitectura, procesos de negocio, datos, API, inteligencia artificial y operación.

Documento de ingeniería
Edición: 10 de noviembre de 2026
Base: revisión del repositorio y documentación disponible

Una visión de extremo a extremo para entender qué resuelve SobraCero, cómo se conectan sus componentes y cuáles son sus límites operativos actuales.

SobraCero convierte excedentes de alimentos cercanos a su fecha de vencimiento en lotes publicables, reservables y recogibles, con apoyo de pronóstico de ventas y herramientas de IA sujetas a revisión humana.

Contenido

- 1. Resumen ejecutivo y propósito
- 2. Actores y capacidades del producto
- 3. Recorrido principal de negocio
- 4. Funcionalidades por módulo
- 5. Arquitectura y responsabilidades técnicas
- 6. Modelo de datos y reglas relevantes
- 7. Superficie de API
- 8. Inteligencia artificial y pronóstico
- 9. Seguridad, privacidad e imágenes
- 10. Interfaz, idiomas y experiencia
- 11. Ejecución local y despliegue
- 12. Pruebas y evidencia disponible
- 13. Límites y prioridades técnicas
- 14. Glosario y fuentes

La guía distingue lo observado en código/documentación de lo que requiere preparación adicional. Una ruta API no implica que exista el mismo control en la interfaz.

1. Resumen ejecutivo y propósito

SobraCero es una aplicación web de rescate de alimentos. El comercio registra tiendas, productos y lotes que desea vender con descuento o donar; el público descubre lotes activos, reserva una cantidad y coordina su recogida con un código. La plataforma ayuda al comercio a organizar inventario mediante historial de ventas, estimación de riesgo de desperdicio, textos promocionales y lectura asistida de fechas impresas en imágenes.

El problema tiene dos caras: reducir alimentos que podrían terminar como desperdicio y facilitar que consumidores accedan a productos disponibles a menor precio. La aplicación modela este flujo como una transacción de inventario y reserva: la reserva descuenta stock y su retiro debe ser confirmado por el comercio propietario.

Qué es y qué no es

- Es una plataforma web con cuentas de comercio y cliente, gestión de tienda/catálogo/lotes, escaparate público y operaciones de reserva y recogida.
- Incluye analítica que calcula riesgo con existencias, ventas recientes, días restantes y, cuando se valida, un modelo estacional por día de semana.
- La IA aporta sugerencias de texto y fechas. No determina precios, stock ni decisiones críticas de inventario; el OCR requiere confirmación humana.
- No es un sistema certificado de inocuidad alimentaria ni valida automáticamente que un producto sea seguro para consumir.

El repositorio describe un MVP y una pila de desarrollo local. No hay despliegue público de producción verificado ni evaluación de IA con datos reales representativos.

2. Actores y capacidades del producto

Actor	Objetivo	Capacidades principales
Comercio	Recuperar valor de productos cercanos a vencer.	Crear tiendas/productos/lotes; cargar imágenes; revisar reservas; confirmar recogidas; importar ventas; consultar riesgo y promociones.
Cliente	Encontrar alimentos disponibles y reservarlos.	Explorar lotes, filtrar por vencimiento, descuento, categoría, tipo de comercio y ubicación; reservar; obtener código/referencia; cancelar reserva.
Administrador	Supervisar la plataforma y mantener el pronóstico.	Consultar conteos agregados y entrenar/evaluar un modelo mediante endpoints administrativos.

No se encontró un rol separado de banco de alimentos. La función de donación registra una cantidad ligada a un lote; no equivale a un flujo integral de organizaciones beneficiarias y trazabilidad de entrega.

3. Recorrido principal de negocio

Paso	Acción	Resultado y control
1. Identidad	Registro o inicio como comercio/cliente.	Token de acceso breve; renovación en cookie HttpOnly respaldada por Redis.
2. Oferta	El comercio crea tienda, producto y lote.	El backend valida rol y propiedad de tienda/producto.
3. Descubrimiento	El cliente explora escaparate y detalle.	Lotes activos, con stock positivo y vencimiento futuro; filtros por negocio/ubicación.
4. Reserva	El cliente solicita una cantidad.	Actualización transaccional de stock y entrega de referencia/código único.
5. Recogida	El comercio propietario confirma con referencia y código.	Se verifican rol, propiedad, estado y código; limitación por IP.
6. Seguimiento	Trabajos caducan lotes/reservas; comercio revisa riesgo/promos.	Celery Beat programa revisión aproximadamente cada 60 segundos.

Las reservas activas son únicas por par cliente/lote. La restricción también existe en la base de datos. Cancelación y confirmación cambian el ciclo de vida; no se debe asumir una segunda reserva activa idéntica.

4. Funcionalidades por módulo

4.1 Cuentas y sesiones

El registro admite perfiles de comercio o cliente. El backend verifica credenciales, protege contraseñas mediante bcrypt y emite token de acceso de corta duración (15 minutos por defecto, según la guía de seguridad). El token se mantiene en memoria del navegador; un refresh token se almacena en cookie HttpOnly y se gestiona con Redis para rotar, renovar y revocar sesiones. La API ofrece registro, inicio, renovación, cierre y consulta del usuario actual.

4.2 Comercios, tiendas y productos

Una cuenta de comercio administra tiendas con tipo (restaurante o supermercado), descripción y coordenadas. Cada tienda contiene productos y categorías; cada lote asocia tienda y producto. El control de propiedad se realiza en el servidor, no solo ocultando controles visuales.

4.3 Publicación y escaparate

Un lote contiene título, descripción, cantidad, precio original en centavos, descuento, vencimiento, estado e imagen opcional. El escaparate entrega lotes activos con disponibilidad y fecha futura, ordenados por vencimiento. Admite filtros por tiempo hasta vencer, descuento mínimo, categoría, tipo de establecimiento y límites geográficos.

4.4 Reservas, recogida, cancelación y donación

El cliente reserva una cantidad disponible y recibe referencia y código de ocho caracteres. El comercio propietario confirma la recogida. Las rutas permiten cancelar una reserva propia o cancelar un lote junto con reservas activas. La donación registra cantidad asociada a un lote. Conflictos de stock y duplicados activos se tratan como reglas de negocio.

4.5 Fotografías de lotes

Se aceptan JPEG, PNG y WebP estáticos; se contrasta el contenido decodificado con el tipo declarado, se limita tamaño (5 MiB) y dimensiones, se normaliza orientación EXIF y se vuelve a codificar para retirar metadatos. El almacenamiento usa clave aleatoria en bucket compatible con S3 privado; la lectura usa URL firmada de vida corta.

4.6 Promociones

El backend genera tres variantes de texto promocional y conserva variante, idioma y prompt para trazabilidad. Cuando Redis/Celery está disponible el trabajo se encola; existe fallback en proceso. El texto depende del proveedor IA y no reemplaza reglas de negocio ni revisión del comercio.

4.7 Fecha de vencimiento asistida por imagen

El comercio propietario o administrador solicita escanear la imagen privada del lote. La imagen se descarga, sanitiza y envía al proveedor configurado; la respuesta se valida por esquema e incluye fecha impresa, propuesta y confianza. Una sugerencia legible queda pendiente; si no supera el umbral, se marca ilegible. Solo una confirmación explícita modifica la fecha del lote. Rechazar no altera la fecha.

4.8 Ventas, riesgo y forecast

El comercio envía ventas por API/CSV o carga 60 días deterministas de muestra. El CSV admite fecha, unidades y si hubo promoción, con máximo 1 MiB y 90 filas. El riesgo se calcula por lote a partir de stock, historial y tiempo hasta vencer. La respuesta incluye ventas esperadas, unidades expuestas, nivel/puntaje y versión del modelo. El historial es privado para propietario y administradores.

4.9 Administración y salud

La API ofrece conteos administrativos y endpoint para entrenar/evaluar pronóstico. El entrenamiento es solo para administradores y limitado por frecuencia. El panel frontend revisado presenta conteos y una serie de pronóstico; el endpoint de entrenamiento existe en backend, pero la interfaz revisada no demuestra un control visual para ejecutarlo.

- GET /health es liveness: confirma que el proceso responde.
- GET /ready comprueba consulta a la base y ping a Redis; no prueba S3 ni proveedor IA.

5. Arquitectura y responsabilidades técnicas

El navegador consume Next.js; el frontend llama a FastAPI mediante un cliente compartido. El backend usa SQLAlchemy para persistencia, Redis para sesiones/caché/cuota/colas, almacenamiento compatible con S3 para fotos y Celery para tareas asíncronas. La base principal declarada es PostgreSQL.

Componente	Tecnología observada	Responsabilidad
Presentación	Next.js 15, React 19, TypeScript, Tailwind	Páginas localizadas, formularios, sesión, mercado y paneles.
Cliente API	app-frontend/lib/api.ts, fetch, Zod	Centraliza HTTP, token en memoria, refresh y locale.
API	FastAPI, Pydantic	Validación, identidad, permisos, coordinación y OpenAPI.
Aplicación/dominio	DTOs, puertos, casos de uso, entidades	Operaciones, invariantes y contratos; arquitectura parcial.
Infraestructura	SQLAlchemy, Redis, adaptadores S3 e IA	Persistencia, sesiones, repositorios, proveedores y predicción.
Procesamiento	Celery worker y Beat	Promociones y expiración periódica; Redis como broker.
Entorno local	PostgreSQL 16, Redis 7, LocalStack	Dependencias para desarrollo mediante Docker Compose.

Flujo de una solicitud

1) El navegador presenta una acción y el cliente adjunta credencial e idioma. 2) FastAPI valida contrato e identidad. 3) La ruta autoriza rol/propiedad y delega lógica. 4) Repositorios/adaptadores acceden a Postgres, Redis, S3 o IA. 5) La respuesta tipada vuelve a la interfaz. Para tareas demoradas se publica trabajo en Redis y Celery lo ejecuta de forma independiente.

La separación domain/application/infrastructure existe, pero es parcial: app/main.py aún contiene rutas y lógica de varios flujos y no todas las rutas pasan por un caso de uso. Es más exacto hablar de backend en transición a capas que de Clean Architecture plenamente aplicada.

Principios de diseño observables

- Inversión de dependencias en áreas donde los casos de uso dependen de puertos y la infraestructura implementa contratos.
- DTOs y validación de esquemas en fronteras: Pydantic en API y respuestas estructuradas de IA.
- Adaptadores por proveedor y almacenamiento intercambiable S3-compatible; proveedor falso determinista para pruebas.
- Restricciones relacionales respaldan estados, unicidad de historial y reservas activas.

6. Modelo de datos y reglas relevantes

Entidad	Propósito	Datos representativos
User	Identidad y rol; posee tiendas.	email, password_hash, role, is_admin
Shop	Unidad operativa.	owner_id, name, business_type, coordenadas
Product	Catálogo y fuente de ventas.	shop_id, name, category
Lot	Oferta con stock y vencimiento.	quantity, price cents, discount, status, image_key
Reservation	Apartado para un cliente.	lot_id, user_id, quantity, pickup_code, status
Donation	Cantidad donada desde un lote.	lot_id, quantity, created_at
Promotion	Variante de texto generada.	lot_id, language, variant, prompt
SalesHistory	Ventas diarias por producto.	product_id, sales_date, units_sold, had_promotion
WastePrediction	Resultado de riesgo.	risk_score, units_at_risk, model_version
ExpiryScanSuggestion	Propuesta OCR revisable.	lot_id, expires_on, confidence, status
SalesForecastModel	Artefacto y evaluación de modelos.	version, MAE, estado, artefacto

Relaciones centrales: User 1:N Shop; Shop 1:N Product y Lot; Product 1:N Lot y SalesHistory; Lot 1:N Reservation, Donation, Promotion y ExpiryScanSuggestion. WastePrediction relaciona lotes/productos.

Reglas destacadas: lotes y reservas tienen estados permitidos; ventas no admiten unidades negativas ni dos filas del mismo producto/fecha; reserva activa por cliente/lote usa índice único; el score se limita a 0-100; las entradas tienen límites de esquema.

7. Superficie de API

La API está versionada bajo /api/v1 y publica esquema interactivo en /docs en ejecución local. El cliente web comunica el idioma en Accept-Language. Las rutas se muestran relativas al prefijo indicado, excepto /health y /ready.

Área	Métodos y rutas	Uso / restricciones
Salud	GET /health; GET /ready	Liveness y comprobación de Postgres/Redis.
Auth	POST /auth/register, /login, /refresh, /logout; GET /auth/me	Refresh cookie y Redis.
Tiendas	GET/POST /shops; GET/POST /shops/{shop_id}/products	Escrituras de comercio con control de propiedad.
Ventas	POST /products/{product_id}/sales-history[/csv]; POST /shops/{shop_id}/sales-history/sample	Importación; CSV hasta 1 MiB y 90 filas.
Análisis	GET /shops/{shop_id}/waste-risk; GET /shops/{shop_id}/forecast	Riesgo y serie forecast por tienda.
Lotes	GET/POST /lots; GET /lots/{lot_id}; POST /lots/{lot_id}/image, /donate, /cancel	Escaparate filtrable; escrituras protegidas.
Reservas	POST /lots/{lot_id}/reserve; POST /reservations/{reservation_id}/cancel, /pickup	Cliente reserva/cancela; comercio confirma.
Promos	POST /lots/{lot_id}/generate-promos; GET /lots/{lot_id}/promos	Genera y lee variantes.
OCR	POST/GET /lots/{lot_id}/expiry-scans; POST .../{suggestion_id}/confirm|reject	Sugerencia con decisión humana.
Admin	POST /admin/models/sales-forecast/train; GET /admin/report	Entrenamiento restringido y estadísticas.

OpenAPI es la fuente para campos exactos, parámetros y códigos de respuesta; esta guía no sustituye el contrato generado.

8. Inteligencia artificial y pronóstico

8.1 Adaptación y control de IA

StructuredAIProvider solicita respuestas estructuradas y las valida con modelos Pydantic. La composición selecciona adaptador: mock determinista, Anthropic o adaptadores compatibles con API de OpenAI (OpenAI, Ollama, Groq, OpenRouter según README). Puede manejar texto o imagen para lectura de fecha. Aplica timeout, hasta dos intentos con pausa, caché temporal, cuota diaria por usuario y registro de latencia/tokens.

La clave de caché incorpora usuario, idioma, versión de prompt, proveedor/modelo, tipo de respuesta y hash de imagen. El modo mock facilita pruebas y demos, pero no mide capacidades remotas. Credenciales y modelos se configuran por entorno; nunca se deben colocar secretos en variables frontend públicas.

8.2 Cálculo del riesgo

La función de dominio calcula ventas esperadas como suma de pronósticos diarios; unidades en riesgo = máximo(0, stock - ventas esperadas); score = 100 por unidades en riesgo dividido por stock, redondeado y limitado a 0-100. El nivel y recomendación se presentan junto con el modelo utilizado. La IA no fija el número ni modifica inventario.

8.3 Entrenamiento estacional

El candidato weekday-seasonal compara una base de promedio móvil con factores por día de la semana. Usa ventanas continuas de siete días, fechas ordenadas, hasta 365 días recientes y holdout temporal (80% inicial para ajuste, 20% posterior para evaluación). Se requieren al menos 14 muestras de entrenamiento y 7 de evaluación. Solo se activa si el MAE de holdout es estrictamente inferior al baseline. En otro caso permanece baseline-weighted-moving-average.

No hay dataset productivo representativo ni benchmark publicado. Las métricas son un mecanismo de comparación, no prueba de calidad predictiva para cada comercio o región. Los datos sintéticos verifican el flujo, no el impacto real.

9. Seguridad, privacidad e imágenes

Control	Implementación documentada	Precaución
Contraseñas/sesión	bcrypt; JWT breve; refresh HttpOnly rotativo con Redis.	HTTPS y cookies Secure son requisitos de producción.
Autorización	Roles business/customer/admin y controles de propiedad.	La API es autoridad; mantener pruebas de acceso cruzado.
Imágenes	JPEG/PNG/WebP, inspección real, límites, EXIF, clave aleatoria y bucket privado.	IA transmite imágenes al proveedor configurado.
Rate limit	Sliding window en proceso, usualmente por IP.	No comparte estado entre réplicas; usar gateway distribuido.
CORS/config	Orígenes configurables y validación de configuración de producción.	Restringir orígenes y guardar secretos en gestor.
Errores	Manejadores tipados para algunos errores.	No todas las rutas comparten contrato único.
Readiness	/ready revisa DB y Redis.	No verifica S3, IA ni tareas Celery.

Amenazas prácticas a controlar

- Aislamiento de cuentas: impedir acceso a tiendas/lotes de otro propietario y proteger rotación/revocación de tokens.
- Abuso de inventario: mantener reserva atómica, no sobre-vender stock, limitar intentos y proteger códigos.
- Carga maliciosa: validar bytes reales/dimensiones y limitar tamaño antes de almacenar o enviar archivos a IA.
- Exposición: ventas, prompts, imágenes y códigos requieren permisos y retención apropiados.
- Operación: validar dependencias, TLS, backups/restauración, claves, logs y respuesta a incidentes.

La guía de seguridad del proyecto declara que no es pentest ni garantía de producción. No se verificó HTTPS público, política integral de headers, limitador distribuido, recuperación automatizada ni flujo de eliminación de cuenta/archivos.

10. Interfaz, idiomas y experiencia

La interfaz usa Next.js App Router con rutas /en y /es, navegación next-intl, middleware y diccionarios en ambos idiomas. El cliente comunica idioma mediante Accept-Language. La preferencia de tema admite sistema, claro y oscuro. Las rutas principales incluyen portada/mercado, login, registro, tienda del comercio, alta/detalle de lotes y administración.

app-frontend/lib/api.ts centraliza llamadas, mantiene access token en memoria y renueva ante 401 mediante cookie. Hay esquema Zod explícito para sugerencias de fecha y tipos TypeScript para modelos API; no se debe asumir que todas las respuestas se validan en tiempo de ejecución con Zod.

- El escaparate comunica descuento, stock y fecha de vencimiento.
- El comercio necesita pasos claros para crear oferta, revisar reserva, preparar pedido y confirmar recogida.
- La fecha detectada por IA se presenta como propuesta pendiente, no como modificación automática.
- Inglés y español están disponibles; eso no garantiza traducción de datos de negocio o salida de proveedor.

11. Ejecución local y despliegue

Docker Compose agrupa PostgreSQL 16, Redis 7, LocalStack S3, FastAPI, Celery worker, scheduler/Beat y Next.js. Los volúmenes de DB y LocalStack conservan datos locales. Compose contiene credenciales de desarrollo: no es una plantilla de exposición pública.

Servicio	Uso	Dependencia
Frontend	Interfaz local en localhost:3000/en o /es.	Backend HTTP
Backend	API en localhost:8000; OpenAPI en /docs.	Postgres, Redis, LocalStack e IA según configuración
PostgreSQL	Datos transaccionales y métricas/modelos.	Backend y worker
Redis	Sesiones, caché/cuota IA y broker Celery.	Backend, worker y scheduler
LocalStack	Emulación S3 para fotos en desarrollo.	Backend
Worker/Beat	Promociones y expiración periódica.	Redis y PostgreSQL

Puesta en marcha documentada

- Asegurar .env local basado en .env.example; nunca sobrescribir el existente ni compartir su contenido.
- Ejecutar docker compose up --build -d desde la raíz.
- Cargar datos demo con docker compose exec backend python seed_data.py.
- Verificar http://localhost:8000/health y /ready.
- Abrir http://localhost:3000/en o /es y revisar http://localhost:8000/docs.

PostgreSQL se publica en el puerto local 5433; LocalStack usa 4566. docker compose down conserva volúmenes. Eliminar datos locales requiere revisar el objetivo y autorización, no forma parte del apagado normal.

Configuración sensible

El operador gobierna APP_ENVIRONMENT, DATABASE_URL, REDIS_URL, JWT_SECRET, CORS_ORIGINS, REFRESH_COOKIE_SECURE, configuración S3, proveedor/llaves/modelos IA, timeout, tokens máximos, cuota y caché. Los valores demo son solo para desarrollo. NEXT_PUBLIC_API_URL es pública y compilada en el frontend: no debe contener secretos.

12. Pruebas y evidencia disponible

El repositorio contiene 19 módulos de pruebas backend (conteo de archivos test_*.py observado), con suites de dominio, autenticación, endpoints, migraciones, CSV, riesgo, promociones, almacenamiento, worker y modelos. README documenta cobertura mínima de 70% para domain/application. En frontend se observan tests de componentes/API/preferencia de tema y pruebas Playwright de idioma/tema y flujo publicar-reservar-recoger.

Evidencia	Qué respalda	Qué no demuestra
Unitarias dominio/aplicación	Reglas deterministas y errores.	Calidad de proveedor IA en producción.
Integración backend	Contratos, persistencia y roles.	Escalabilidad multi-réplica.
Tests frontend	Interacciones y estado de UI/API.	Servicios externos reales disponibles.

Evidencia	Qué respalda	Qué no demuestra
Playwright	Demo con API determinista e idioma/tema.	DB, S3, Celery o IA reales.
Evaluación forecast	MAE de candidato vs baseline temporal.	Precisión comercial sin datos reales.

Esta tarea fue de documentación; no se ejecutó el conjunto de pruebas ni se levantó la aplicación. La presencia de tests se verificó en el inventario, no como resultado de una corrida durante la elaboración del PDF.

13. Límites conocidos y prioridades técnicas

La documentación de auditoría existente registra una fotografía anterior a varias capacidades hoy descritas por README y visibles en código (por ejemplo refresh sessions, importación CSV, OCR, entrenamiento temporal y middleware locale). Para evitar contradicciones, este PDF contrasta las capacidades con código actual y docs de API/arquitectura/seguridad/evaluación; se recomienda actualizar docs/audit.md en una revisión posterior.

Prioridad	Brecha / precaución	Siguiente paso
P0 Producción	Compose usa credenciales de desarrollo; no se verificó HTTPS.	Entorno administrado, TLS, secretos, CORS y backups probados.
P0 Seguridad	Rate limit en memoria por proceso; proxy/IP y rutas requieren política distribuida.	Limitador edge con confianza de proxy explícita y límites por usuario/IP.
P0 Privacidad	Imágenes pueden enviarse a proveedor externo; retención/residencia dependen del operador.	Evaluar proveedor, minimización, retención y respuesta a incidentes.
P1 Arquitectura	Parte de rutas/lógica permanece en main.py y services.py.	Extraer casos de uso preservando comportamiento con pruebas.
P1 Contratos	Errores no siguen aún un único envelope message_key.	Centralizar excepciones sin filtrar errores de proveedor.
P1 UI	Endpoint de entrenamiento existe, pero el panel observado no evidencia control para usarlo.	Confirmar alcance y conectar UI si se requiere.
P1 Datos	Sin benchmark real de forecast ni evaluación IA con proveedor real.	Definir conjunto de aceptación, etiquetas, precisión, latencia y costo.
P2 Operación	Ready no verifica almacenamiento/IA; baja y eliminación de archivos no validada.	Definir health, retención, borrado, backup y restauración.
P2 Docs	Auditoría inicial no está alineada con funcionalidades actuales.	Mantener matriz de estado y documentación versionada.

Lectura de riesgos por ingeniería

La reserva y actualización de inventario son transaccionalmente sensibles: preservar atomicidad e idempotencia bajo concurrencia. El OCR es apoyo humano, no control de inocuidad. El forecast debe indicar modelo/version y no presentar MAE sintético como promesa comercial. La capa de seguridad de despliegue debe completarse antes de exponer API pública.

14. Glosario y fuentes consultadas

Término	Significado
Lote	Cantidad de producto publicada con vencimiento, stock, precio y descuento.
Reserva	Apartado de stock para cliente, con estado y código de recogida.
MAE	Error absoluto medio; menor es mejor en la comparación usada.
Baseline	Método de referencia (promedio móvil ponderado) que el candidato debe superar.
Holdout temporal	Fechas posteriores reservadas para evaluar sin entrenar sobre ellas.
URL firmada	Enlace firmado y temporal para leer objeto privado.
DTO / puerto / adaptador	Objeto de transferencia, contrato de aplicación e implementación externa.

Fuentes internas revisadas

- README.md: recorrido de producto, configuración local, funcionalidades y pruebas recomendadas.
- docs/architecture.md y docs/api.md: componentes, modelo de datos, flujos y endpoints.
- docs/evaluation.md y docs/security.md: metodología de evaluación, controles y limitaciones.
- docs/deployment.md y docs/demo-script.md: servicios, operación y recorrido de demostración.
- app-backend/app/main.py, presentation/routers/auth.py, models.py, llm.py, domain/services/waste_risk.py e infrastructure/prediction/weekday_sales_forecast.py.
- app-frontend/app/[locale]/*, lib/api.ts, middleware.ts, i18n/routing.ts y package.json.
- docker-compose.yml y estructura de pruebas backend/frontend.

Documento descriptivo elaborado desde el repositorio disponible el 10 de noviembre de 2026. No constituye certificación de seguridad, asesoría legal, validación sanitaria ni evaluación estadística independiente.